Especificação completa — Agentes autônomos de monitoramento veicular

Projeto: AITrack + DataDrivr + Dirijabem
Baseline auditada: `5593ec5` (`checkpoint-2026-09-16-1801-antes-codex`)
Data: 13/10/2026
Repositório canônico: `/home/pasteurjr/progreact/aitrack`
Backend ativo: `/home/pasteurjr/progreact/aitrack/aitrackdatadrivr`

1. Resumo executivo

O produto será um sistema de agentes autônomos que substitui a rotina operacional de um monitor humano de frota. O agente deverá observar telemetria, entender contexto, decidir o que fazer, executar comunicações, aguardar respostas, reavaliar o caso, escalar quando necessário e registrar sua prestação de contas.

O detector não é o agente. O detector fornece os sentidos. A IA externa é o componente de interpretação e decisão. A fila de ações executa as decisões. A memória mantém contexto e histórico. Os canais de comunicação dão ao agente capacidade de agir no mundo real.

Fluxo obrigatório:

```
telemetria → evento → contexto/memória → decisão do agente → ação → resposta
       ↑                                                          ↓
       └──────────── acompanhamento e reavaliação ────────────────┘
```

2. O que já está pronto

2.1 Fontes de dados e simuladores

Componente	Estado	Evidência
Tracker via TCP	**Pronto para teste**	`aitrackdatadrivr/simulator.py` gera 10 veículos na porta 9000.
Protocolos Tracker	**Parcial/ pronto**	Parsers Suntech e Queclink ativos; Maxtrack existe, mas não está habilitado no fluxo principal.
Rotas Tracker	**Parcial**	Há configuração de rotas e perfis, porém o heading ainda não acompanha a geometria da rota de forma realista.
Replay Dirijabem	**Pronto para teste**	`dirijabem_simulator.py` reproduz viagens e grava nas tabelas Dirijabem.
Viagens sintéticas Dirijabem	**Pronto para teste**	Perfis excellent/good/average/poor/aggressive disponíveis.
Simulador contínuo Dirijabem	**Pronto para teste**	`dirijabem_continuous_simulator.py` gera motoristas virtuais continuamente.
API de simulação	**Parcial**	`server/dirijabem_api.py` possui início, parada, posição, rota e status.

2.2 Detecção comportamental

Já existem detecções no `server/behavioral_engine.py` para:

- `harsh_accel` — aceleração brusca;
- `harsh_brake` — frenagem brusca;

- `sharp_turn` — curva brusca;
- `speeding` — excesso de velocidade.

O motor recebe posições Tracker, calcula comportamento e pode persistir eventos. A estrutura é uma base válida para os agentes, mas ainda não é adequada para operação contínua: usa amostras em vez de tempo real decorrido, o score não se recupera corretamente, a memória não tem limite/sincronização e a persistência é síncrona.

2.3 Backend de monitores

Já existem:

- migrations de monitores, veículos monitorados, análises e alertas;
- CRUD de monitores;
- associação e remoção de veículos;
- ativação/desativação de monitor;
- endpoints de `acknowledge`, `resolve` e `dismiss`;
- `server/monitor_engine.py` com análise baseada em regras;
- estrutura para salvar análises e alertas.

O motor não é iniciado pelo `run.py`. A análise ainda é rule-based e não chama Claude ou outro LLM. O backend ainda não possui runtime de agente, memória de conversa, fila de ações, executor de canais ou acompanhamento de respostas.

2.4 Frontend

O App 2 possui telas e componentes para monitores, alertas, eventos e veículos. A API oferece parte das operações necessárias.

O App 2 ainda utiliza `mockData`, `mockAlerts`, `mockEvents`, `mockMonitors` e `mockVehiclesInMonitors` em fluxos importantes. A interface, portanto, aparenta possuir mais comportamento do que o backend efetivamente entrega.

2.5 Infraestrutura e documentação

Já existem documentação de handoff, auditoria, instruções de execução e plano inicial. O baseline está preservado em `5593ec5`. A suíte de testes ativa ainda é insuficiente para validar o motor comportamental, os monitores e os agentes.

3. O que ainda não existe

3.1 Inteligência autônoma

Faltam:

- definição persistida de missão e responsabilidade de cada agente;
- nível de autonomia configurável;
- memória operacional e memória histórica;
- loop observar–decidir–agir–acompanhar;
- decisão estruturada por IA externa;
- validação da decisão antes da execução;
- ferramentas que a IA pode chamar;
- política de limites e permissões;
- reavaliação após uma ação;
- fechamento baseado em evidência;
- justificativa e auditoria de cada decisão.

3.2 Ações no mundo real

Faltam:

- fila durável de ações;
- executor assíncrono com tentativas e backoff;
- WhatsApp ou SMS;
- e-mail;
- ligação telefônica automática;
- confirmação de entrega e leitura quando o provedor suportar;
- recebimento e interpretação de respostas;
- escalonamento para contato secundário;
- templates por severidade e destinatário;
- limites de frequência e janela de silêncio;
- cancelamento/pausa de ação pendente.

Essas integrações exigem credenciais e contratação/configuração de provedores externos. O código atual não pode alegar que liga ou envia mensagem enquanto esses conectores não estiverem configurados e testados.

4. Definição de um agente monitor

Cada agente deverá ser uma entidade configurável com os campos abaixo:

```
{
  "name": "Monitor de Segurança",
  "mission": "reduzir risco de acidentes e proteger motoristas",
  "observes": ["harsh_brake", "harsh_accel", "speeding", "collision_suspected"],
  "goals": ["identificar_risco", "orientar", "escalar", "confirmar_resolucao"],
  "autonomy_level": 3,
  "allowed_tools": [
    "get_vehicle_context",
    "get_driver_history",
    "send_driver_message",
    "send_owner_message",
    "make_phone_call",
    "create_incident",
    "escalate_incident"
  ],
  "forbidden_tools": ["delete_event", "alter_telemetry", "close_without_evidence"],
  "follow_up_minutes": 10
}
```

A IA recebe a missão, o evento e o contexto. Ela não recebe acesso livre ao banco nem executa SQL. Ela devolve uma decisão JSON que o sistema valida contra as permissões do agente.

5. Como a IA participa

5.1 Detector: os sentidos

Regras e algoritmos determinísticos detectam fatos mensuráveis: velocidade, aceleração, frenagem, localização, tempo parado, bateria e rota. Esses componentes devem ser previsíveis e testáveis sem LLM.

5.2 IA externa: interpretação e julgamento operacional

A IA externa, inicialmente Claude configurável por `ANTHROPIC_API_KEY` e `ANTHROPIC_MODEL`, deverá:

- interpretar eventos isolados no contexto do histórico;
- distinguir incidente real de possível falso positivo;

- classificar risco e urgência;
- escolher uma ferramenta permitida;
- redigir comunicação apropriada;
- decidir prazo para acompanhamento;
- determinar quando escalar;
- explicar a decisão em linguagem humana;
- responder mensagens recebidas do responsável ou motorista.

A IA não poderá inventar localização, motorista, horário ou evidência. Toda afirmação operacional deverá apontar para dados fornecidos no contexto.

5.3 Runtime: continuidade e responsabilidade

O runtime mantém o agente ativo. Ele consome eventos, agrupa episódios, chama a IA quando necessário, grava a decisão, enfileira ações, processa resultados e agenda o próximo acompanhamento.

5.4 Executor: braços do agente

O executor transforma uma decisão autorizada em ação externa. Cada ação tem status `pending`, `running`, `sent`, `delivered`, `failed`, `cancelled` ou `expired`, tentativas, prazo, resposta do provedor e idempotency key.

6. Loop de operação humana

O comportamento mínimo esperado é:

1. Observar novos eventos.
2. Agrupar eventos do mesmo episódio.
3. Buscar contexto do veículo, motorista, rota e histórico.
4. Avaliar risco por regras.
5. Chamar a IA para interpretação quando houver novidade relevante.
6. Escolher ação conforme autonomia e política.
7. Executar mensagem, ligação, tarefa ou escalonamento.
8. Registrar exatamente o que foi feito.
9. Aguardar resposta ou prazo de acompanhamento.
10. Reavaliar a situação com os novos dados.
11. Repetir, escalar ou encerrar com evidência.

Um alerta sem acompanhamento não é considerado atuação de agente.

7. Níveis de autonomia

Nível	Comportamento	Exemplos
0 — observador	Detecta e registra	Telemetria experimental.
1 — assistente	Prepara decisão e mensagem para aprovação	Casos ambíguos ou sensíveis.
2 — supervisionado	Executa mensagens de baixo risco e cria tarefas	Excesso de velocidade pontual.
3 — autônomo	Comunica, acompanha e escala sem aprovação individual	Bateria baixa, reincidência de risco.
4 — crítico	Atua imediatamente, exige confirmação posterior	Colisão provável ou emergência operacional.

O nível não é permissão absoluta: a matriz de política pode restringir uma ferramenta específica mesmo em nível alto.

8. Agentes iniciais

8.1 Agente de segurança e direção

Missão: reduzir condução perigosa e risco de acidente. Observa frenagem, aceleração, curvas, velocidade, reincidência e possível colisão. Orienta motorista, avisa proprietário, abre incidente, liga e escala conforme gravidade.

8.2 Agente de operação e disponibilidade

Missão: manter veículos operacionais. Observa bateria, GPS, perda de comunicação, veículo parado e ignição. Solicita verificação, cria tarefa de manutenção e acompanha normalização.

8.3 Agente de rota e jornada

Missão: verificar cumprimento de rota e horário. Observa geofence, desvio, parada não prevista, horário anormal e viagem Dirijabem. Avisa gestor e registra justificativa.

8.4 Agente de manutenção

Missão: antecipar falhas e reduzir indisponibilidade. Usa histórico de eventos, quilometragem quando disponível e sinais de bateria/dispositivo. Cria tarefa, define prazo e cobra conclusão.

8.5 Agente de atendimento ao proprietário

Missão: responder perguntas e fornecer situação atual. Consulta somente dados autorizados, explica eventos e informa ações já tomadas. Não pode alterar dados críticos sem aprovação.

9. Matriz de resposta

Situação	Decisão padrão	Comunicação	Acompanhamento
Evento isolado de baixa gravidade	Registrar	Painel	Não abrir ocorrência se normalizar.
Reincidência em janela curta	Orientar	Motorista e gestor	Reavaliar em 10 minutos.
Risco alto persistente	Abrir incidente	Mensagem ao motorista e proprietário	Ligar se não houver resposta.
Possível colisão	Prioridade crítica	Ligação, localização e mensagem	Escalar imediatamente.
Perda de sinal	Criar tarefa técnica	Operação/manutenção	Verificar retorno e escalar.
Desvio de rota	Solicitar justificativa	Motorista e gestor	Encerrar somente com justificativa.

10. Guardrails obrigatórios

- No máximo um incidente aberto por agente, veículo e episódio.
- Cooldown configurável por tipo de evento e destinatário.
- Limite diário de mensagens e ligações.
- Janela de silêncio configurável, exceto nível crítico.
- Idempotência em toda ação externa.

- IA sem acesso direto ao banco ou a segredos.
- Mensagens sempre baseadas em evidências do contexto.
- Ações críticas configuráveis para aprovação humana.
- Toda decisão, tentativa, resposta e escalonamento auditados.
- Fallback para regras quando a IA ou o provedor falhar.
- Nenhum caso encerrado sem normalização observável ou confirmação autorizada.

11. Modelo mínimo de dados

As migrations deverão acrescentar, sem destruir dados existentes:

- `agent_definitions`: missão, autonomia, ferramentas e políticas;
- `agent_runs`: cada ciclo de decisão;
- `agent_memory`: fatos resumidos e contexto operacional;
- `incidents`: episódio consolidado e estado;
- `agent_actions`: ação, destino, canal, status, tentativas e idempotência;
- `agent_messages`: mensagens enviadas/recebidas;
- `agent_escalations`: níveis e contatos acionados;
- `agent_audit_log`: decisão, versão de prompt, modelo e evidências.

12. Contrato de decisão da IA

A resposta deverá ser JSON validado por schema:

```
{
  "risk": "low|medium|high|critical",
  "assessment": "string",
  "evidence_ids": ["event-123"],
  "actions": [
    {
      "tool": "send_owner_message",
      "recipient_id": "owner-9",
      "message": "string",
      "priority": "normal|high|urgent"
    }
  ],
  "follow_up": {"after_minutes": 10, "condition": "no_response"},
  "close_condition": "string",
  "confidence": 0.0
}
```

JSON inválido, ferramenta não permitida, destinatário inexistente ou evidência ausente bloqueia a ação e retorna ao fallback rule-based.

13. Estado de prontidão

Pronto para começar a implementação

- fontes Tracker e Dirijabem simuláveis;
- detecção inicial de comportamento;
- banco e migrations iniciais de monitores;
- endpoints básicos de alerta e monitor;
- frontend de acompanhamento;
- baseline versionado e documentação de handoff.

Parcial e precisa ser corrigido

- score e detecção temporal;
- realismo do simulador Tracker;
- imports e fallback da API;
- integração frontend/API;
- inicialização do motor;
- uso de Dirijabem pelo motor;
- testes automatizados.

Ausente e necessário para agentes autônomos

- runtime de agentes;
- cliente de IA externa;
- memória e episódios;
- fila/executor de ações;
- conectores de comunicação;
- respostas recebidas;
- follow-up e escalonamento;
- auditoria e políticas de autonomia.

14. Critério de pronto do produto

O produto só será considerado pronto quando um cenário simulado demonstrar, sem intervenção manual no código:

1. Tracker ou Dirijabem produzir evento.
2. Agente consolidar o episódio.
3. Agente consultar contexto.
4. IA devolver decisão estruturada, ou fallback funcionar.
5. Ação ser enfileirada e executada por conector fake/real.
6. Resultado ser registrado.
7. Agente acompanhar a situação.
8. Ausência de resposta provocar escalonamento.
9. Normalização ou confirmação encerrar o caso.
10. Painel exibir todo o histórico.